

Day 07 – Linux File System Hierarchy & Scenario-Based Practice

You'll create a file called:

None

```
day-07-linux-fs-and-scenarios.md
```

This task helps you understand:

- Where Linux stores files
- How DevOps engineers troubleshoot systems
- How to investigate services, logs, CPU usage, and permissions

Part 1 — Linux File System Hierarchy

Step 1: Create Your Notes File

Run:

None

```
touch day-07-linux-fs-and-scenarios.md
```

Verify:

None

```
ls
```

Example Output

None

```
day-07-linux-fs-and-scenarios.md
```

Explanation

- `touch` creates a new empty file
 - `ls` lists files in the current directory
-

Step 2: Open the File

Use nano editor:

None

```
nano day-07-linux-fs-and-scenarios.md
```

SECTION 1 — Core Linux Directories

1. `/` (Root Directory)

Command

None

```
ls -l /
```

Example Output

None

```
drwxr-xr-x  bin
```

```
drwxr-xr-x  etc
drwxr-xr-x  home
drwxr-xr-x  var
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /
total 60
lrwxrwxrwx   1 root root      7 Apr 20 08:46 bin -> usr/bin
drwxr-xr-x   5 root root  4096 Apr 21 05:24 boot
drwxr-xr-x  16 root root  3420 May 21 14:28 dev
drwxr-xr-x 108 root root  4096 May 20 06:00 etc
drwxr-xr-x   3 root root  4096 May 20 06:00 home
lrwxrwxrwx   1 root root      7 Apr 20 08:46 lib -> usr/lib
lrwxrwxrwx   1 root root      9 Apr 20 08:46 lib64 -> usr/lib64
drwx-----  2 root root 16384 Apr 21 05:21 lost+found
drwxr-xr-x   2 root root  4096 Apr 21 05:18 media
drwxr-xr-x   2 root root  4096 Apr 21 05:18 mnt
drwxr-xr-x   2 root root  4096 Apr 21 05:18 opt
dr-xr-xr-x 180 root root      0 May 21 14:28 proc
drwx-----  4 root root  4096 May 20 06:00 root
drwxr-xr-x  31 root root   920 May 21 14:29 run
lrwxrwxrwx   1 root root      8 Apr 20 08:46 sbin -> usr/sbin
drwxr-xr-x   6 root root  4096 Apr 21 05:25 snap
drwxr-xr-x   2 root root  4096 Apr 21 05:18 srv
dr-xr-xr-x  13 root root      0 May 21 14:28 sys
drwxrwxrwt  12 root root   240 May 21 14:29 tmp
drwxr-xr-x  12 root root  4096 Apr 21 05:18 usr
drwxr-xr-x  13 root root  4096 May 20 06:00 var
```

Explanation

- `/` is the top-level directory
- Everything in Linux starts from here

I would use this when...

“I need to navigate the complete Linux file system.”

2. `/home`

Command

```
None  
ls -l /home
```

Example Output

```
None  
drwxr-xr-x  ubuntu  
drwxr-xr-x  devops
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /home  
total 4  
drwxr-x--- 7 ubuntu ubuntu 4096 May 21 14:30 ubuntu
```

Explanation

- Stores normal users' files and folders
- Each user gets a separate home directory

I would use this when...

"I need to access user files or scripts."

3. /root

Command

```
None  
ls -l /root
```

Example Output

None

```
backup.sh  
notes.txt
```

```
ubuntu@ip-172-31-32-12:~$ sudo ls -l /root  
total 4  
drwx----- 3 root root 4096 May 20 06:00 snap
```

Explanation

- Home directory of the root user
- Only accessible by administrators

I would use this when...

“I am logged in as root and managing server-level tasks.”

4. /etc

Command

None

```
ls -l /etc
```

Example Output

None

```
hostname  
passwd  
ssh  
systemd
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /etc
total 916
drwxr-xr-x 5 root root      4096 Apr 21 05:19 ModemManager
drwxr-xr-x 2 root root      4096 Apr 21 05:20 PackageKit
drwxr-xr-x 4 root root      4096 Apr 21 05:18 X11
drwxr-xr-x 4 root root      4096 Apr 21 05:24 acpi
-rw-r--r-- 1 root root      4027 Nov  4 2025 adduser.conf
drwxr-xr-x 2 root root      4096 Apr 21 05:21 alternatives
drwxr-xr-x 2 root root      4096 Apr 21 05:19 apparmor
drwxr-xr-x 9 root root      4096 Apr 21 05:20 apparmor.d
drwxr-xr-x 3 root root      4096 Apr 21 05:20 appport
drwxr-xr-x 8 root root      4096 Apr 21 05:25 apt
-rw-r--r-- 1 root root     2553 Feb 13 12:16 bash.bashrc
-rw-r--r-- 1 root root        45 Sep 28 2025 bash_completion
drwxr-xr-x 2 root root      4096 Apr 21 05:20 bash_completion.d
-rw-r--r-- 1 root root      367 Mar 10 15:03 bindresvport.blacklist
drwxr-xr-x 2 root root      4096 Apr 15 18:32 binfmt.d
drwxr-xr-x 3 root root      4096 Apr 21 05:18 ca-certificates
-rw-r--r-- 1 root root     6174 Apr 21 05:18 ca-certificates.conf
drwxr-xr-x 4 root root      4096 Apr 21 05:18 chrony
```

Explanation

- Stores system configuration files
- Very important directory for DevOps engineers

I would use this when...

“I need to modify system or application configurations.”

5. /var/log

Command

None

```
ls -l /var/log
```

Example Output

None

syslog
auth.log
kern.log

```
ubuntu@ip-172-31-32-12:~$ ls -l /var/log
total 1988
lrwxrwxrwx 1 root    root          39 Apr 21 05:18 README -> ../../usr/share/doc/systemd/README.logs
-rw-r--r-- 1 root    root          736 Apr 21 05:21 alternatives.log
drwx----- 3 root    root          4096 May 20 06:00 amazon
-rw-r----- 1 root    adm           0 May 20 06:00 apport.log
drwxr-xr-x 2 root    root          4096 Apr 21 05:25 apt
-rw-r----- 1 syslog  adm          20550 May 21 14:38 auth.log
-rw-rw---- 1 root    utmp           0 Apr 21 05:20 btmp
drwxr-x--- 2 _chrony _chrony       4096 May 20 06:00 chrony
-rw-r----- 1 root    adm          8995 May 21 14:28 cloud-init-output.log
-rw-r----- 1 syslog  adm        674605 May 21 14:28 cloud-init.log
drwxr-xr-x 2 root    root          4096 Apr 16 17:50 dist-upgrade
-rw-r----- 1 root    adm         49899 May 21 14:28 dmesg
-rw-r----- 1 root    adm         49378 May 21 05:53 dmesg.0
-rw-r----- 1 root    adm         15067 May 20 16:01 dmesg.1.gz
-rw-r----- 1 root    adm         15146 May 20 10:57 dmesg.2.gz
-rw-r----- 1 root    adm         15696 May 20 06:00 dmesg.3.gz
-rw-r--r-- 1 root    root        27279 Apr 21 05:25 dpkg.log
drwxr-sr-x+3 root    systemd-journal 4096 May 20 06:00 journal
-rw-r----- 1 syslog  adm        305551 May 21 14:29 kern.log
drwxr-xr-x 2 landscape landscape    4096 May 20 06:01 landscape
-rw-rw-r-- 1 root    utmp        292292 May 21 14:29 lastlog
drwx----- 2 root    root          4096 May 20 06:00 private
-rw-r----- 1 syslog  adm        765022 May 21 14:40 syslog
drwxr-xr-x 2 root    root          4096 May 21 05:53 sysstat
drwxr-x--- 2 root    adm          4096 May 21 14:39 unattended-upgrades
-rw-rw-r-- 1 root    utmp          5760 May 21 14:29 wtmp
```

Explanation

- Stores system and application logs
- Critical for troubleshooting

I would use this when...

"I need to investigate errors or service failures."

6. /tmp

Command

None

```
ls -l /tmp
```

Example Output

None

```
temp1.txt
```

```
systemd-private
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /tmp
total 0
drwx----- 2 root root 40 May 21 14:28 snap-private-tmp
drwx----- 3 root root 60 May 21 14:28 systemd-private-f611a9987356433689d820a6e870e42f-ModemManager.service-Wv63hG
drwx----- 3 root root 60 May 21 14:28 systemd-private-f611a9987356433689d820a6e870e42f-chrony.service-iAnLeu
drwx----- 3 root root 60 May 21 14:28 systemd-private-f611a9987356433689d820a6e870e42f-irqbalance.service-6D2h1v
drwx----- 3 root root 60 May 21 14:28 systemd-private-f611a9987356433689d820a6e870e42f-polkit.service-yvFLfQ
drwx----- 3 root root 60 May 21 14:28 systemd-private-f611a9987356433689d820a6e870e42f-systemd-logind.service-IAZRy4
ubuntu@ip-172-31-32-12:~$
```

Explanation

- Stores temporary files
- Files may be deleted after reboot

I would use this when...

“I need temporary storage during script execution.”

Additional Important Directories

7. /bin

Command

None

```
ls -l /bin
```

Example Output

None

```
bash
```

```
ls
```

```
cat
```

```
cp
```

```
ubuntu@ip-172-31-32-12:~$ ls /bin
3cpio          glib-compile-schemas  ldattach      py3versions    strip
VGAAuthService 'gnu['               ldd            pybabel         stty
'['            gnuarch              less           pybabel-python3 su
aa-enabled     gnub2sum             lessecho      pydoc3          su-rs
aa-exec        gnubase32            lessfile      pydoc3.14       sudo
aa-features-abi gnubase64            lesskey       pygettext3      sudoedit
acpi_listen    gnubasename          lesspipe      pygettext3.14   sudo.rs
acpidbg        gnubasenc            lexgrog       pygmentize      sudo.ws
add-apt-repository gnucap              libnetcfg     pyhtmlizer3     sudoedit-rs
addpart        gnuchcon             link           pyserial-miniterm sudoreplay.ws
addr2line      gnuchgrp             linux-boot-prober sum
apport-bug     gnuchmod             linux-check-removal switch_root
apport-cli     gnuchown             linux-run-hooks sync
apport-collect gnucksum             linux-update-symlinks systemctl
apport-unpack  gnucmm              linux-version systemd-ac-power
appstreamcli   gnuccp              linux32        systemd-analyze
apropos        gnucsplit           linux64        systemd-ask-password
apt            gnuclib             ln             systemd-cat
apt-add-repository gnuclib             lnstat        systemd-cgls
apt-cache      gnudd               loadkeys      systemd-cgtop
apt-cdrom      gnudd               loadunimap    systemd-confext
apt-config     gnuclib             locale        systemd-creds
apt-get        gnuclib             locale-check  systemd-cryptenroll
apt-mark       gnuclib             localedef     systemd-cryptsetup
```

Explanation

- Contains essential Linux commands

I would use this when...

“I need basic system commands.”

8. /usr/bin

Command

None

```
ls -l /usr/bin | head
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /usr/bin | head
total 178848
-rwxr-xr-x 1 root root 709512 Feb 5 09:55 3cpio
-rwxr-xr-x 1 root root 133768 Mar 4 21:48 VGAuthService
lrwxrwxrwx 1 root root 28 Mar 30 16:50 [ -> ../lib/cargo/bin/coreutils/[
-rwxr-xr-x 1 root root 14720 Apr 8 17:26 aa-enabled
-rwxr-xr-x 1 root root 14720 Apr 8 17:26 aa-exec
-rwxr-xr-x 1 root root 14720 Apr 8 17:26 aa-features-abi
-rwxr-xr-x 1 root root 15000 Dec 1 10:22 acpi_listen
-rwxr-xr-x 1 root root 1622 Apr 13 08:12 acpidbg
-rwxr-xr-x 1 root root 25004 Apr 2 02:23 add-apt-repository
```

Example Output

None

```
python3
git
vim
docker
```

Explanation

- Stores user-level command binaries

I would use this when...

“I need installed applications and tools.”

9. /opt

Command

None

```
ls -l /opt
```

Example Output

None

```
google  
microsoft
```

```
ubuntu@ip-172-31-32-12:~$ ls -l /opt  
total 0  
ubuntu@ip-172-31-32-12:~$
```

Explanation

- Stores third-party software

I would use this when...

“I install external applications manually.”

Hands-On Practice Tasks

Task 1: Find Largest Log Files

Command

None

```
du -sh /var/log/* 2>/dev/null | sort -h | tail -5
```

```
ubuntu@ip-172-31-32-12:~$ du -sh /var/log/* 2>/dev/null | sort -h | tail -5
52K    /var/log/dmesg.0
300K   /var/log/kern.log
664K   /var/log/cloud-init.log
748K   /var/log/syslog
17M    /var/log/journal
ubuntu@ip-172-31-32-12:~$
```

Example Output

None

```
5M    /var/log/auth.log
12M   /var/log/syslog
30M   /var/log/kern.log
```

Explanation

Command Part	Meaning
<code>du -sh</code>	Shows file sizes
<code>2>/dev/null</code>	Hides permission errors
<code>sort -h</code>	Sorts by size
<code>tail -5</code>	Shows largest 5 files

Why Important?

DevOps engineers use this to identify:

- Disk space issues
- Huge log files
- Misbehaving services

Task 2: View Hostname Config

Command

None

```
cat /etc/hostname
```

```
ubuntu@ip-172-31-32-12:~$ cat /etc/hostname
ip-172-31-32-12
```

Example Output

None

```
ip-172-31-20-10
```

Explanation

- `cat` displays file contents
 - `/etc/hostname` stores machine hostname
-

Task 3: Check Home Directory

Command

None

```
ls -la ~
```

```

ubuntu@ip-172-31-32-12:~$ ls -la ~
total 80
drwxr-x--- 7 ubuntu ubuntu 4096 May 21 14:30 .
drwxr-xr-x 3 root   root   4096 May 20 06:00 ..
-rw----- 1 ubuntu ubuntu 1721 May 21 06:08 .bash_history
-rw-r--r-- 1 ubuntu ubuntu 220  Feb 13 12:16 .bash_logout
-rw-r--r-- 1 ubuntu ubuntu 3771 Feb 13 12:16 .bashrc
drwx----- 2 ubuntu ubuntu 4096 May 20 06:01 .cache
drwx----- 3 ubuntu ubuntu 4096 May 20 06:20 .config
-rw----- 1 ubuntu ubuntu  20  May 20 16:37 .lessht
drwxrwxr-x 3 ubuntu ubuntu 4096 May 20 06:09 .local
-rw-r--r-- 1 ubuntu ubuntu  807 Feb 13 12:16 .profile
drwx----- 2 ubuntu ubuntu 4096 May 20 06:00 .ssh
-rw----- 1 ubuntu ubuntu  774 May 20 06:12 .viminfo
-rw-rw-r-- 1 ubuntu ubuntu    0 May 21 14:30 day-07-linux-fs-and-scenarios.md
drwxrwxr-x 2 ubuntu ubuntu 4096 May 21 06:07 day06-file-io
-rw-rw-r-- 1 ubuntu ubuntu 9763 May 20 07:06 linux-practice.md
-rw-rw-r-- 1 ubuntu ubuntu 9746 May 20 17:08 linux-troubleshooting-runbook.md
-rw----- 1 ubuntu ubuntu 2981 May 20 16:46 linux-troubleshooting-runbook.md.save

```

Example Output

```

None
.bashrc
.ssh
Downloads
Documents

```

Explanation

Option	Meaning
-l	Long listing
-a	Show hidden files
~	Current user home

Part 2 — Scenario-Based Practice

Scenario 1 — Service Not Starting

Problem

nginx service failed after reboot.

Step 1: Check Service Status

Command

```
None  
systemctl status nginx
```

Example Output

```
None  
Active: failed
```

Why?

Shows whether service is:

- running
 - stopped
 - failed
-

Step 2: Check Logs

Command

```
None  
journalctl -u myapp -n 50
```

Example Output

None

Error: Port already in use

Why?

Displays last 50 log lines for the service.

Step 3: Check Boot Enablement

Command

None

```
systemctl is-enabled myapp
```

Example Output

None

```
enabled
```

Why?

Checks if service starts automatically on reboot.

Step 4: Restart Service

Command

None

```
sudo systemctl restart myapp
```

Why?

Attempts to recover service after fixing issues.

What You Learned

Troubleshooting flow:

1. Check status
 2. Read logs
 3. Verify startup settings
 4. Restart service
-

Scenario 2 — High CPU Usage

Problem

Server is slow.

Step 1: Monitor Live Processes

Command

None

```
top
```

Example Output

None

```
PID USER  %CPU COMMAND
2345 root  92.1 java
```

Why?

Shows real-time CPU usage.

Press:

None

q

to exit.

Step 2: Sort Processes by CPU

Command

None

```
ps aux --sort=-%cpu | head -10
```

Example Output

None

```
root 2345 92.1 java
```

Why?

Shows top CPU-consuming processes.

Step 3: Get Process Details

Command

None

```
ps -p 2345 -f
```

Example Output

None

```
java application.jar
```

Why?

Provides details about the problematic process.

Step 4: Optional — Kill Process

Command

None

```
kill -9 2345
```

Why?

Stops a stuck or harmful process.

What You Learned

How to:

- identify CPU bottlenecks
 - locate bad processes
 - investigate performance issues
-

Scenario 3 — Finding Service Logs

Problem

Developer wants Docker logs.

Step 1: Check Service Status

Command

None

```
systemctl status docker
```

Why?

Confirms service state.

Step 2: View Recent Logs

Command

None

```
journalctl -u docker -n 50
```

Example Output

None

```
Docker daemon started
```

Why?

Shows latest service logs.

Step 3: Follow Logs Live

Command

None

```
journalctl -u docker -f
```

Why?

Streams logs in real time.

Press:

None

`Ctrl + C`

to stop.

What You Learned

systemd services store logs in:

- journald
 - accessible via `journalctl`
-

Scenario 4 — File Permission Issue

Problem

`backup.sh` gives:

None

`Permission denied`

Step 1: Check Permissions

Command

None

`ls -l /home/user/backup.sh`

Example Output

None

```
-rw-r--r--
```

Explanation

No **x** permission means:

✗ not executable

Step 2: Add Execute Permission

Command

None

```
chmod +x /home/user/backup.sh
```

Why?

Adds executable permission.

Step 3: Verify Permissions

Command

None

```
ls -l /home/user/backup.sh
```

Example Output

None

```
-rwxr-xr-x
```

Explanation

x means executable.

Step 4: Run Script

Command

None

```
./backup.sh
```

Example Output

None

```
Backup completed successfully
```

What You Learned

How Linux permissions work:

Symbol	Meaning
--------	---------

r	read
---	------

w	write
---	-------

x	execute
---	---------

Final Summary

Today you learned:

- Linux directory structure
 - Where configs and logs are stored
 - How to troubleshoot services
 - How to investigate CPU problems
 - How Linux permissions work
-

Important DevOps Directories to Remember

Directory	Purpose
<code>/etc</code>	Configuration files
<code>/var/log</code>	Logs
<code>/home</code>	User files
<code>/tmp</code>	Temporary files
<code>/bin</code>	Essential commands
<code>/usr/bin</code>	Installed binaries
<code>/opt</code>	Third-party software

Recommended Practice

Try these on your Linux VM:

```
None
top
journalctl -xe
systemctl list-units --type=service
df -h
free -m
```

These are real DevOps troubleshooting commands used daily.